Manual de Bibliotecas OpenGL en Visual Studio con vcpkg

Guía técnica de instalación global, generación de ejecutables independientes (.exe estáticos) y mantenimiento de paquetes en C++

1. Instalación Global de vcpkg elimina la configuración repetitiva por proyecto

El gestor de paquetes vcpkg de Microsoft descarga, compila e integra automáticamente las bibliotecas de C++ con MSBuild en Visual Studio. Una vez realizada la integración global, cualquier proyecto nuevo configurado en arquitectura **x64** reconoce los archivos de cabecera (`.h` y `.hpp`), enlaza los binarios (`.lib`) y copia las dependencias dinámicas (`.dll`) sin configurar rutas manualmente.

Requisitos de ubicación y sintaxis de consola en Windows

- **Ruta corta obligatoria:** Clona el repositorio oficial microsoft/vcpkg en GitHub en una ruta cercana a la raíz del disco o del Escritorio (como `C:\vcpkg` o `C:\Users\AlanV\Desktop\extension_bibliotecas\vcpkg`). Las rutas anidadas con espacios superan el límite de 260 caracteres de Windows y provocan fallos de extracción (`BUILD_FAILED` en `vcpkg_extract_source_archive`).
- **Diferencia entre terminales:** En **Símbolo del sistema (CMD)** y **PowerShell** los ejecutables locales se invocan con barra invertida (`.\vcpkg.exe`), mientras que en **Git Bash (MINGW64)** se invocan con diagonal normal (`./vcpkg.exe`).

Secuencia de instalación e integración inicial (CMD / PowerShell)

1. **Clonar y compilar el gestor vcpkg:**

   ```
   cd C:\Users\AlanV\Desktop\extension_bibliotecas
   git clone https://github.com/microsoft/vcpkg.git
   cd vcpkg
   .\bootstrap-vcpkg.bat
   ```

2. **Instalar el conjunto completo de bibliotecas para OpenGL (arquitectura de 64 bits por defecto):**

   ```
   .\vcpkg.exe install glfw3 glad glm glew stb assimp
   ```

3. **Vincular vcpkg globalmente con Visual Studio:**

   ```
   .\vcpkg.exe integrate install
   ```

   Al confirmar el mensaje `Applied user-wide integration for this vcpkg root`, reinicia Visual Studio si estaba abierto. No muevas ni renombres la carpeta de `vcpkg` después de este paso (si la cambias de lugar, ejecuta nuevamente `.\vcpkg.exe integrate install` desde la nueva ruta).

2. Catálogo de bibliotecas de OpenGL y directivas #include

Todas las bibliotecas instaladas quedan disponibles de inmediato para cualquier proyecto de C++ en Visual Studio seleccionando la plataforma **x64** en la barra superior:

Paquete vcpkg	Función en proyectos OpenGL	Cabeceras principales en C++	Comando individual
glfw3	Creación de ventanas, contexto OpenGL y captura de teclado/ratón.	`#include <GLFW/glfw3.h>`	`.\vcpkg.exe install glfw3`
glad	Cargador de punteros a funciones de OpenGL moderno (no requiere agregar `glad.c` manualmente).	`#include <glad/glad.h>`	`.\vcpkg.exe install glad`
glm	Matemáticas vectoriales y matriciales (transformaciones 3D, cámaras y proyecciones).	`#include <glm/glm.hpp>` `#include <glm/gtc/matrix_transform.hpp>`	`.\vcpkg.exe install glm`
glew	Cargador clásico de extensiones OpenGL (alternativa a GLAD).	`#include <GL/glew.h>`	`.\vcpkg.exe install glew`
stb	Carga de texturas e imágenes (PNG, JPG, TGA) en una sola cabecera.	`#define STB_IMAGE_IMPLEMENTATION` `#include <stb_image.h>`	`.\vcpkg.exe install stb`
assimp	Importación de modelos y mallas 3D completas (`.obj`, `.fbx`, `.gltf`).	`#include <assimp/Importer.hpp>` `#include <assimp/scene.h>` `#include <assimp/postprocess.h>`	`.\vcpkg.exe install assimp`

3. Creación de un ejecutable independiente (.exe sin archivos .dll)

Por defecto, `vcpkg` instala paquetes dinámicos (triplet `x64-windows`) que generan archivos `.dll` junto a tu ejecutable. Para compilar un **único archivo `.exe` autónomo** que funcione en cualquier otra computadora sin pedir `.dll` ni tener instalado Visual Studio, se utiliza el **enlace estático (*Static Linking*)**.

Paso 3.1: Instalar las versiones estáticas de las bibliotecas

Abre CMD en tu carpeta de vcpkg y añade el sufijo `:x64-windows-static` a cada biblioteca que utilice tu proyecto:

```
.\vcpkg.exe install glfw3:x64-windows-static glad:x64-windows-static glm:x64-windows-static glew:x64-windows-static stb:x64-windows-static assimp:x64-windows-static
```

Paso 3.2: Configurar las propiedades del proyecto en Visual Studio

En el *Explorador de soluciones*, haz clic derecho sobre el nombre de tu proyecto, selecciona **Propiedades** y aplica los ajustes de la siguiente tabla:

Ubicación en Visual Studio	Parámetro u opción	Valor requerido	Propósito técnico
Barra superior / Configuración	Configuración y Plataforma	**Release | x64**	Elimina símbolos de depuración y optimiza el ejecutable final.
Propiedades > vcpkg	Use Static Libraries	**Yes (Sí)**	Indica a `vcpkg` que enlace los archivos `.lib` estáticos en lugar de usar `.dll`.
Propiedades > vcpkg	Use Dynamic CRT	**No**	Usa el runtime estático compatible con `x64-windows-static`.
Propiedades > C/C++ > Generación de código	Biblioteca en tiempo de ejecución (*Runtime Library*)	**Multiproceso (`/MT`)** *(o `/MTd` en Debug)*	Incrusta las librerías base de C++ de Windows (`vcruntime140.dll`) dentro del `.exe`.

Paso 3.3: Compilación y manejo de archivos externos (Shaders, texturas y modelos)

- **Ubicación del ejecutable:** Presiona `Ctrl + Shift + B` (*Compilar solución*). El archivo autónomo se generará dentro de la carpeta de tu solución en `x64\Release\TuProyecto.exe`.
- **Archivos de recursos gráficos:** El enlace estático integra todas las bibliotecas (`glfw3`, `glad`, `assimp`, etc.) dentro del `.exe`. Sin embargo, si tu código abre archivos externos mediante rutas como `"shaders/vertex.glsl"`, `"textures/pared.jpg"` o `"models/nave.obj"`, debes copiar esas carpetas junto al archivo `.exe` respetando la misma estructura de carpetas relativas (o bien declarar el código de tus shaders como cadenas `const char*` directamente dentro de tu archivo `.cpp`).

4. Mantenimiento, limpieza de espacio y desinstalación de bibliotecas

Todos los comandos de administración se ejecutan desde el Símbolo del sistema (CMD) situado en la carpeta donde instalaste vcpkg (`C:\Users\AlanV\Desktop\extension_bibliotecas\vcpkg`):

1. Consultar y desinstalar bibliotecas específicas

- **Ver la lista de todos los paquetes instalados actualmente:**

  ```
  .\vcpkg.exe list
  ```

- **Borrar las bibliotecas dinámicas (`x64-windows`) y sus dependencias internas:**

  ```
  .\vcpkg.exe remove glfw3 glad glm glew stb assimp --recurse
  ```

- **Borrar las bibliotecas estáticas (`x64-windows-static`):**

  ```
  .\vcpkg.exe remove glfw3:x64-windows-static glad:x64-windows-static glm:x64-windows-static glew:x64-windows-static stb:x64-windows-static assimp:x64-windows-static --recurse
  ```

2. Liberar espacio en disco duro sin perder las bibliotecas instaladas

Al compilar paquetes pesados como Assimp, `vcpkg` conserva archivos temporales que pueden ocupar varios gigabytes. Para recuperar espacio sin afectar tus proyectos de Visual Studio:

- **Carpetas que puedes vaciar o borrar con seguridad:** `buildtrees` (código fuente descomprimido y objetos temporales de compilación), `downloads` (archivos `.zip` y `.tar.gz` descargados) y `packages` (caché intermedia de paquetes).
- **Carpeta obligatoria que debes conservar:** Nunca borres la subcarpeta **`installed`**, ya que ahí se guardan los encabezados (`.h`) y binarios (`.lib` / `.dll`) que utiliza Visual Studio.

3. Desinstalación completa de vcpkg de tu computadora

Si en algún momento deseas eliminar el gestor y todas las bibliotecas por completo sin dejar rastro en el sistema:

1. **Desvincular la integración con Visual Studio:**

   ```
   .\vcpkg.exe integrate remove
   ```

2. **Eliminar los archivos:** Borra la carpeta `vcpkg` de tu Escritorio y, para limpiar también la caché binaria global de usuario, elimina la carpeta `C:\Users\AlanV\AppData\Local\vcpkg`.